

RAG-Based Insurance Policy Chatbot - Project Documentation

1. Project Goals

The primary objective of this project is to build a Retrieval-Augmented Generation (RAG) chatbot that assists users in querying complex insurance documents, specifically a life insurance policy, and returns accurate, grounded answers using LLM-based natural language responses.

Objectives:

- Enable users to query large insurance documents interactively.
- Retrieve relevant document chunks using vector-based semantic search.
- Rerank results for relevance using cross-encoders.
- Generate grounded and factual answers using an LLM (e.g., GPT-3.5).
- Avoid hallucination by strictly limiting responses to retrieved content.

2. Data Sources

Document: Principal-Sample-Life-Insurance-Policy.pdf

Type: Official insurance policy booklet

Format: Multi-page PDF with structured legal and benefits-related text

Source: Provided sample dataset for life insurance policy

3. System Architecture

The system is structured in three main layers, following a classic RAG architecture:

3.1 Embedding Layer:

- Text Extraction: Used pdfplumber to extract page-wise text + tables.
- Chunking: Implemented sliding window strategy (chunk_size=500, overlap=50).
- Embedding Model: text-embedding-ada-002 (OpenAI, 1536-dim).
- Storage: Embedded chunks stored in ChromaDB (PersistentClient) with metadata.

3.2 Search & Rerank Layer:

- Search: Vector similarity using ChromaDB (cosine similarity).
- Cache: Implemented cache layer using a separate ChromaDB collection.
- Reranker: Used cross-encoder/ms-marco-MiniLM-L-6-v2 and experimented with stsb-roberta-base.

3.3 Generation Layer:

- LLM: gpt-3.5-turbo via OpenAI's chat API.
- Prompting Strategy: System + user prompts include strict context limits.
- Safety: LLM instructed to say 'I don't know' when unsure.

4. Sample Queries Used

1. What is the claim process for nominees of the policy?
2. How is the policy surrender value calculated, and when is it applicable?
3. Can a member convert their group life insurance to an individual policy after termination?

5. Design Choices

Component - Choice - Justification

Chunking - Sliding window - Best context handling for legal documents

Embedding - OpenAI ADA-002 - High-accuracy semantic vector search

Vector DB - ChromaDB - Lightweight, local, fast

Reranker - CrossEncoder - Boosts relevance after retrieval

LLM - GPT-3.5 - Balanced performance and cost

Cache - ChromaDB - Avoids duplicate retrieval and generation

6. Challenges Faced

- Tables required special parsing (used pdfplumber table detection).
- Chunking balance between size and accuracy.
- Avoiding hallucination by tuning prompts and controlling LLM output.

7. Outcomes

- Functional insurance chatbot using RAG pipeline.
- Successfully answered 3 real-world queries.
- Clean architecture separating retrieval, reranking, and generation.
- Aligned with industry best practices.

8. Deliverables

- Colab notebook (.ipynb)
- 6 screenshots: 3 for retrieval, 3 for generation
- This documentation (.docx)